



UNIVERSIDADE  
ESTADUAL de LONDRINA

---

Marco Antônio Cottorello Henry e Murilo de Souza Neves

## COMPUTAÇÃO AUTÔNOMA

---

LONDRINA - PR

2026

HENRY, Marco Antônio Cottorello. NEVES, Murilo de Souza. **COMPUTAÇÃO AUTÔNOMA**. 2026. 22 folhas. (Graduação em Ciência da Computação) – Universidade Estadual de Londrina, Londrina, 2026.

## **RESUMO**

O crescimento exponencial da complexidade dos sistemas computacionais tornou insustentável o modelo de administração manual de infraestruturas de TI. A Computação Autônoma, formalizada pela IBM em 2001, propõe sistemas capazes de gerenciar a si mesmos por meio de quatro propriedades fundamentais: auto-configuração, auto-cura, auto-otimização e auto-proteção. Este trabalho apresenta o contexto histórico do paradigma, analisa o ciclo de controle MAPE-K e discute aplicações em servidores corporativos, computação em nuvem e Internet das Coisas, demonstrando a autonomia como requisito para a infraestrutura digital contemporânea.

**Palavras-chave:** Computação Autônoma. Auto-gerenciamento. MAPE-K. Propriedades Self-X. Computação em Nuvem. Internet das Coisas.

HENRY, Marco Antônio Cottorello. NEVES, Murilo de Souza **AUTONOMIC COMPUTING**. 2026. 22 folhas. (Graduação em Ciência da Computação) – Universidade Estadual de Londrina, Londrina, 2026.

## **ABSTRACT**

The exponential growth in the complexity of computing systems has made the manual administration model for IT infrastructures unsustainable. Autonomic Computing, formalized by IBM in 2001, proposes systems capable of managing themselves through four fundamental properties: self-configuration, self-healing, self-optimization, and self-protection. This work presents the historical context of the paradigm, analyzes the MAPE-K control loop, and discusses applications in enterprise servers, cloud computing, and the Internet of Things, demonstrating autonomy as a requirement for contemporary digital infrastructure.

**Key words:** Autonomic Computing. Self-management. MAPE-K. Self-X Properties. Cloud Computing. Internet of Things.

## **Lista de Figuras**

|            |                                                                |           |
|------------|----------------------------------------------------------------|-----------|
| <b>3.1</b> | <b>Árvore de propriedades da computação autônoma . . . . .</b> | <b>13</b> |
| <b>4.2</b> | <b>Arquitetura do ciclo de controle MAPE-K . . . . .</b>       | <b>14</b> |

## **Lista de Tabelas**

|            |                                                                          |           |
|------------|--------------------------------------------------------------------------|-----------|
| <b>2.1</b> | <b>Cronologia de projetos influentes em auto-gerenciamento . . . . .</b> | <b>12</b> |
|------------|--------------------------------------------------------------------------|-----------|

## LISTA DE ABREVIATURAS E SIGLAS

|        |                                                                                 |
|--------|---------------------------------------------------------------------------------|
| AC     | <i>Autonomic Computing</i> (Computação Autônoma)                                |
| CPU    | <i>Central Processing Unit</i> (Unidade Central de Processamento)               |
| DDoS   | <i>Distributed Denial of Service</i> (Ataque Distribuído de Negação de Serviço) |
| IBM    | <i>International Business Machines Corporation</i>                              |
| IoT    | <i>Internet of Things</i> (Internet das Coisas)                                 |
| MAPE-K | <i>Monitor, Analyze, Plan, Execute over Knowledge</i>                           |
| SLA    | <i>Service Level Agreement</i> (Acordo de Nível de Serviço)                     |
| SNA    | Sistema Nervoso Autônomo                                                        |
| TCO    | <i>Total Cost of Ownership</i> (Custo Total de Propriedade)                     |
| TI     | Tecnologia da Informação                                                        |
| VM     | <i>Virtual Machine</i> (Máquina Virtual)                                        |
| XaaS   | <i>Everything as a Service</i> (Tudo como um Serviço)                           |

## Sumário

|          |                                                                   |           |
|----------|-------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introdução</b>                                                 | <b>7</b>  |
| <b>2</b> | <b>Histórico e Desenvolvimento da Computação Autônoma</b>         | <b>10</b> |
| 2.1      | PROJETOS PRECURSORES . . . . .                                    | 10        |
| <b>3</b> | <b>Propriedades de Auto-Gerenciamento</b>                         | <b>12</b> |
| <b>4</b> | <b>O Laço de Controle Autônomo MAPE-K</b>                         | <b>13</b> |
| 4.1      | MONITORAMENTO . . . . .                                           | 14        |
| 4.2      | PLANEJAMENTO E ADAPTAÇÃO . . . . .                                | 15        |
| 4.3      | BASE DE CONHECIMENTO . . . . .                                    | 16        |
| <b>5</b> | <b>Aplicações</b>                                                 | <b>17</b> |
| 5.1      | NASA <i>Autonomous NanoTechnology Swarm</i> (ANTS) . . . . .      | 17        |
| 5.2      | IBM <i>Project eLiza</i> : SERVIDORES AUTO-GERENCIÁVEIS . . . . . | 18        |
| 5.3      | GERENCIAMENTO DE INFRAESTRUTURAS DE CLOUD COMPUTING . . . . .     | 19        |
| 5.4      | SISTEMAS PERVASIVOS E INTERNET DAS COISAS (IoT) . . . . .         | 20        |
| <b>6</b> | <b>Estado da Arte: Avanços, Metodologias e Desafios</b>           | <b>21</b> |
| 6.1      | PRINCIPAIS DESCOBERTAS E METODOLOGIAS . . . . .                   | 21        |
| 6.2      | LACUNAS E DESAFIOS EM ABERTO . . . . .                            | 22        |
| <b>7</b> | <b>Conclusão</b>                                                  | <b>24</b> |
|          | <b>REFERÊNCIAS</b>                                                | <b>25</b> |

## 1 INTRODUÇÃO

A trajetória da computação moderna é marcada por uma transição contínua da simplicidade para a hiper-complexidade. Se nas décadas de 1960 e 1970 a computação era dominada por mainframes isolados e tarefas de processamento em lote, a virada do milênio consolidou uma realidade distinta. A convergência da Lei de Moore, que permitiu um aumento exponencial no poder de processamento, com a onipresença da conectividade de rede, transformou os sistemas computacionais em infraestruturas globais. Hoje, as aplicações não residem mais em uma única máquina, mas em arquiteturas distribuídas que abrangem servidores em nuvem, dispositivos de borda e uma miríade de componentes heterogêneos que precisam interagir em tempo real [1].

Essa evolução, embora tenha possibilitado inovações disruptivas em todos os setores da sociedade, gerou um efeito colateral crítico: a “crise da complexidade”. À medida que os sistemas se tornaram mais interconectados, o número de variáveis envolvidas em sua configuração, manutenção e otimização cresceu de forma não linear. Ambientes corporativos contemporâneos frequentemente integram hardware de múltiplos fornecedores, diferentes sistemas operacionais, middleware complexo e aplicações legadas [2]. Gerenciar todos esses equipamentos exige profissionais altamente qualificados, cujas tarefas diárias são consumidas por atividades repetitivas de instalação, ajuste de desempenho, segurança e recuperação de falhas [3].

O problema central reside no fato de que o gerenciamento desses sistemas ainda é, em grande parte, manual e dependente da intervenção humana. Estudos indicam que o custo de manutenção e administração de sistemas de TI pode representar até 80% do custo total de propriedade (TCO), superando largamente o investimento inicial em hardware e software [4]. Além disso, a falibilidade humana torna-se o elo mais fraco da corrente: a maioria das interrupções de serviço em grandes centros de dados é causada por erros de configuração cometidos por administradores sobrecarregados [5]. Estamos atingindo um limiar onde a complexidade das interações entre os componentes de software ultrapassa a capacidade cognitiva humana de prever comportamentos e mitigar riscos em tempo hábil [2].

Diante desse cenário de insustentabilidade, a IBM lançou em outubro de 2001 um manifesto intitulado “*Autonomic Computing: It’s Time Systems Take Care of Themselves*” [3]. Este documento foi um divisor de águas, argumentando que a solução para a crise de complexidade não residia em ferramentas de gerenciamento mais potentes, mas sim em uma mudança na natureza dos próprios sistemas. A proposta central da Computação Autônoma é inspirada no Sistema Nervoso Autônomo (SNA) do corpo humano. O SNA regula funções vitais como os batimentos cardíacos, a respiração e a temperatura corporal sem que a mente consciente precise intervir [4]. Se um indivíduo precisar correr, o SNA aumenta a frequência cardíaca automaticamente; se a temperatura externa cai, ele induz o tremor para gerar calor.



A Computação Autônoma busca mimetizar essa biologia, criando sistemas capazes de gerenciar a si mesmos de acordo com diretrizes de alto nível estabelecidas pelos administradores [5]. Em vez de configurar manualmente cada parâmetro de um banco de dados, o administrador define apenas os objetivos de negócio (como tempos de resposta ou níveis de disponibilidade), e o sistema se encarrega de ajustar sua infraestrutura interna para atingir tais metas. Esse modelo propõe a liberação dos profissionais de TI das tarefas rotineiras e de baixo nível, permitindo que eles se concentrem em decisões estratégicas e de alto valor agregado [2].

Para operacionalizar essa visão, o modelo autônomo é sustentado por quatro pilares fundamentais, conhecidos como propriedades *Self-CH* [3, 4]:

- Auto-configuração (Self-configuration): Refere-se à capacidade do sistema de se instalar e configurar automaticamente seguindo políticas definidas. Componentes novos devem ser integrados de forma transparente, ajustando-se dinamicamente ao ambiente existente sem necessidade de reinicializações manuais ou ajustes finos exaustivos [2].
- Auto-cura (Self-healing): O sistema deve ser capaz de detectar, diagnosticar e isolar falhas de software e hardware. Após a detecção, ele deve iniciar ações corretivas, minimizando o tempo de inatividade e garantindo a continuidade do serviço [5].
- Auto-otimização (Self-optimization): Consiste na busca contínua pela eficiência máxima. O sistema monitora constantemente seu desempenho e uso de recursos, redistribuindo cargas de trabalho para garantir que os níveis de serviço contratados (SLAs) sejam atendidos de forma econômica [4].
- Auto-proteção (Self-protection): O sistema deve defender-se proativamente contra ataques maliciosos ou falhas em cascata, detectando padrões de intrusão e antecipando problemas antes que comprometam a integridade dos dados [3].

A implementação técnica dessas propriedades geralmente se baseia no ciclo de controle conhecido como MAPE-K (*Monitor, Analyze, Plan, Execute over Knowledge*). Este loop de feedback contínuo permite que o sistema observe seu próprio estado e o do ambiente, analise discrepâncias, planeje as mudanças e as execute [1]. Tudo isso é suportado por uma base de conhecimento que contém modelos do sistema, históricos de comportamento e políticas de gerenciamento.

Apesar da clareza teórica do manifesto da IBM, a implementação plena da Computação Autônoma enfrenta desafios significativos. O primeiro é de ordem técnica: como garantir que um sistema que se auto-ajusta não entre em estados instáveis ou oscilatórios? [5]. A criação de algoritmos de controle robustos para ambientes distribuídos é uma tarefa complexa que exige rigor formal. Além disso, a falta de padrões de interoperabilidade entre fornecedores dificulta a criação de orquestradores universais [2].

Outro obstáculo crítico é a confiança. Administradores hesitam em delegar decisões críticas a algoritmos automáticos, especialmente em sistemas onde falhas resultam

em perdas financeiras ou riscos à segurança [2]. A transição é, portanto, gradual; começa com recomendações para humanos e evolui para a automação total com o amadurecimento tecnológico [5].

Em suma, a Computação Autônoma não é apenas uma conveniência técnica, mas uma necessidade imperativa para a sobrevivência da infraestrutura digital moderna. Ao transferir a carga administrativa do humano para a máquina, ela promete viabilizar a próxima era da computação, onde a complexidade será gerida pela inteligência intrínseca do próprio sistema [3, 4].

Este trabalho está organizado da seguinte forma: a Seção 2 apresenta o histórico e desenvolvimento da Computação Autônoma, abrangendo os projetos precursores, as propriedades *Self-X* e o ciclo de controle MAPE-K. A Seção 3 discute aplicações concretas do paradigma, tanto em contextos clássicos quanto em cenários contemporâneos como computação em nuvem e Internet das Coisas. A Seção 4 analisa o estado da arte, com as principais metodologias emergentes e os desafios em aberto. Por fim, a Seção 5 apresenta as conclusões do trabalho.

## 2 HISTÓRICO E DESENVOLVIMENTO DA COMPUTAÇÃO AUTÔNOMA

A necessidade de sistemas auto-gerenciáveis não é um fenômeno exclusivo da computação moderna. Um paralelo histórico instrutivo pode ser traçado com a indústria telefônica da década de 1920. Naquele período, o crescimento exponencial no uso do telefone exigia um número cada vez maior de operadores humanos para gerenciar as centrais de comutação manuais. As preocupações com a possível escassez de operadores qualificados para atender à demanda eram sérias. A solução foi o desenvolvimento das centrais automáticas de comutação, que eliminaram a necessidade de intervenção humana nessa tarefa [6]. A computação autônoma representa, em essência, uma resposta análoga ao mesmo problema estrutural, agora na escala e complexidade dos sistemas de informação contemporâneos.

O termo *autonomic computing* foi criado pela IBM em 2001. A palavra “autônomo” tem origem na biologia: no corpo humano, o sistema nervoso autônomo regula funções inconscientes como o tamanho da pupila, as funções digestivas, a frequência respiratória e a dilatação dos vasos sanguíneos, sem que a mente consciente precise intervir [6]. A computação autônoma busca intervir nos sistemas computacionais de maneira análoga, reduzindo o envolvimento humano na administração cotidiana. Embora o termo tenha sido formalizado pela IBM, os conceitos subjacentes ao auto-gerenciamento já vinham sendo explorados em diferentes contextos ao longo dos anos anteriores.

### 2.1 PROJETOS PRECURSORES

O campo da computação autônoma foi moldado por uma série de projetos independentes que, sem coordenação explícita, convergiram para os mesmos princípios de auto-gestão [6].

O primeiro projeto notável foi o **Situational Awareness System (SAS)**, iniciado em 1997 pela agência de defesa norte-americana DARPA, como parte do programa *Small Units Operations* (SUO). Seu objetivo era criar dispositivos pessoais de comunicação e localização para soldados em campo de batalha. O sistema deveria propagar autonomamente informações de status entre os dispositivos, adaptar-se às mudanças de topologia da rede e ajustar frequências e largura de banda de acordo com as condições ambientais — inclusive com equipamentos de interferência inimiga em operação. O projeto pioneirou o roteamento ad-hoc descentralizado e adaptativo como forma de auto-gerenciamento em ambientes adversos, lidando com redes que podiam chegar a 10.000 nós móveis [6].

Em 2000, a DARPA lançou o programa **DASADA** (*Dynamic Assembly for Systems Adaptability, Dependability, and Assurance*), cujo objetivo era desenvolver tecnologia

para sistemas críticos com alta confiabilidade e adaptabilidade. Este projeto introduziu a abordagem orientada a arquitetura para o auto-gerenciamento, com a noção de *probes* e *gauges* para monitorar o sistema e um mecanismo de adaptação para disparar mudanças com base nos dados coletados [6]. Essa abordagem viria a influenciar diretamente a arquitetura MAPE-K, descrita na Seção 4.

Em outubro de 2001, a IBM formalizou o conceito com seu manifesto de computação autônoma [3]. A proposta central era que sistemas computacionais complexos deveriam possuir propriedades autônomas, ou seja, deveriam ser capazes de gerenciar independentemente as tarefas regulares de manutenção e otimização, reduzindo assim a carga de trabalho dos administradores. Além de cunhar o termo, a IBM sistematizou as quatro propriedades centrais de um sistema auto-gerenciável, discutidas na Seção 3.

Em 2003, a DARPA iniciou o programa **Self-Regenerative Systems (SPS)**, com foco em sistemas de computação militares capazes de manter funcionalidades críticas mesmo diante de erros não intencionais ou ataques. A abordagem incluía a geração de múltiplas versões funcionalmente equivalentes do software, tornando qualquer ataque incapaz de comprometer simultaneamente uma fração significativa das versões do programa. Um modelo de confiança era utilizado para afastar a computação de recursos propensos a causar dano, e uma arquitetura de replicação tolerante a intrusões de larga escala era desenvolvida em paralelo [6].

Por fim, em 2005, a NASA iniciou o projeto **Autonomous NanoTechnology Swarm (ANTS)**, cujo objetivo era explorar o cinturão de asteroides com um enxame de mil pequenas espaçonaves. Dado que entre 60% e 70% das naves eram esperadas a ser perdidas ao entrar no cinturão, as sobreviventes deveriam colaborar autonomamente para completar a missão. O projeto foi motivado também pelo longo atraso de comunicação entre sondas em espaço profundo e o controle em Terra, tornando o gerenciamento manual impraticável. A NASA já havia utilizado comportamentos autônomos nas missões DS1 (*Deep Space 1*) e Mars Pathfinder, e o projeto ANTS representou um passo além nessa direção [6].

A Tabela 2.1 apresenta uma síntese cronológica desses projetos e suas contribuições para o campo.

**Tabela 2.1** – Cronologia de projetos influentes em auto-gerenciamento

| Projeto             | Ano  | Origem | Contribuição principal                                 |
|---------------------|------|--------|--------------------------------------------------------|
| SAS                 | 1997 | DARPA  | Roteamento ad-hoc descentralizado e adaptativo         |
| DASADA              | 2000 | DARPA  | Probes e gauges para monitoramento arquitetural        |
| Computação Autônoma | 2001 | IBM    | Formalização do termo e das quatro propriedades Self-X |
| SPS                 | 2003 | DARPA  | Sistemas auto-regenerativos resistentes a ataques      |
| ANTS                | 2005 | NASA   | Enxame autônomo para exploração espacial               |

### 3 PROPRIEDADES DE AUTO-GERENCIAMENTO

As quatro propriedades fundamentais de um sistema autônomo, conforme sistematizadas pela IBM, são conhecidas como propriedades *Self-X*. Cada uma aborda uma dimensão distinta do auto-gerenciamento [2, 6].

**Auto-configuração (*Self-Configuration*):** Um sistema autônomo configura-se de acordo com objetivos de alto nível. Isso significa que o administrador especifica *o que é* desejado, não *como* alcançá-lo. O sistema instala-se e ajusta-se automaticamente de acordo com as necessidades da plataforma e dos usuários [6].

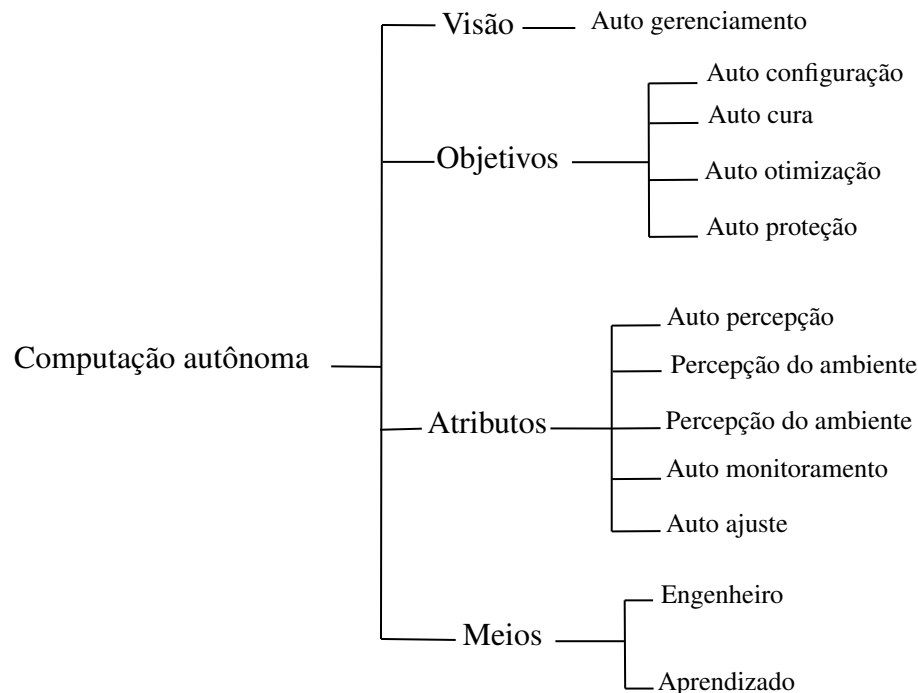
**Auto-otimização (*Self-Optimization*):** O sistema otimiza proativamente o uso de seus recursos, podendo decidir por iniciativa própria realizar mudanças para melhorar o desempenho ou a qualidade do serviço. Essa propriedade distingue-se pelo seu caráter *pró-ativo*: o sistema não aguarda uma solicitação externa para agir, ele antecipa a necessidade [6].

**Auto-cura (*Self-Healing*):** O sistema detecta e diagnostica problemas, que podem variar de erros de baixo nível em chips de memória a falhas de software em serviços de diretório. Sempre que possível, o sistema tenta corrigir o problema — por exemplo, alternando para um componente redundante ou instalando atualizações de software — garantindo que o processo de correção não introduza novos problemas [6]. A tolerância a falhas é um aspecto central: o sistema deve ser *reativo* a falhas ou a seus primeiros sinais.

**Auto-proteção (*Self-Protection*):** O sistema protege-se tanto de ataques maliciosos quanto de usuários que inadvertidamente realizam alterações prejudiciais. O sistema ajusta-se autonomamente para alcançar segurança, privacidade e proteção de dados [6]. Além de reagir a ameaças, o sistema deve ser capaz de antecipar brechas de segurança e impedir que ocorram, exibindo características *proativas*.

Essas propriedades guardam estreita relação com as características de agentes de software identificadas por Wooldridge e Jennings, que incluem: (i) *autonomia*, operar sem intervenção humana direta mantendo controle sobre suas ações e estado interno; (ii) *habilidade social*, interagir com outros agentes por meio de algum tipo de linguagem de comunicação; (iii) *reatividade*, perceber o ambiente e responder a mudanças em tempo hábil; e (iv) *pró-atividade*, tomar iniciativa para atingir objetivos de forma dirigida por metas [6].

**Figura 3.1** – Árvore de propriedades da computação autônoma



#### 4 O LAÇO DE CONTROLE AUTÔNOMO MAPE-K

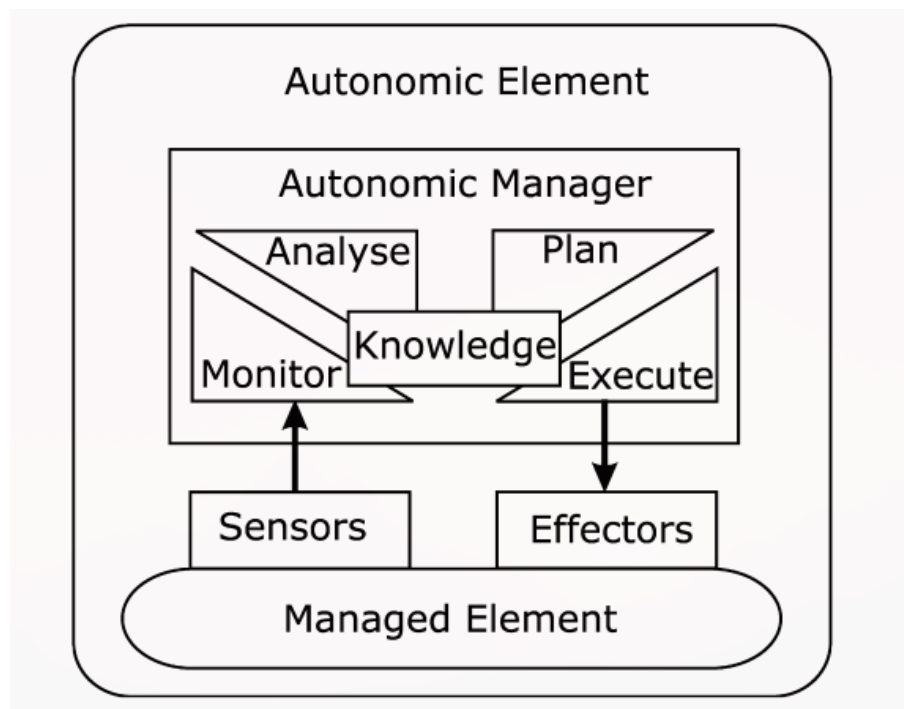
Para operacionalizar as propriedades Self-X, a IBM propôs um modelo de referência para laços de controle autônomos denominado MAPE-K — sigla para *Monitor, Analyse, Plan, Execute over Knowledge* [1, 6]. Este modelo descreve o ciclo funcional de um *Elemento Autônomo*, composto por um *Gerenciador Autônomo* acoplado a um *Elemento Gerenciado*.

O *Elemento Gerenciado* pode ser qualquer recurso de software ou hardware ao qual se deseja conferir comportamento autônomo: um servidor web, um banco de dados, um sistema operacional, um cluster de máquinas, uma rede com ou sem fio, uma CPU, etc. [6]. A interação entre o gerenciador e o elemento gerenciado ocorre por meio de dois tipos de componentes de interface:

- **Sensores (*Sensors/Probes/Gauges*):** Coletam informações sobre o elemento gerenciado.

Para um servidor web, isso pode incluir tempo de resposta a requisições de clientes, uso de rede, disco, CPU e memória [6]. Os *gauges* podem filtrar, agregar e processar os dados brutos coletados pelos *probes* antes de reportá-los ao gerenciador, e podem residir em máquinas distintas do elemento gerenciado.

- **Efetores (*Effectors*):** Implementam mudanças no elemento gerenciado. Mudanças podem ser de granularidade grossa — como adicionar ou remover servidores de um cluster de servidores web — ou de granularidade fina — como alterar parâmetros de configuração individuais em um servidor [6].



**Figura 4.2** – Arquitetura do ciclo de controle MAPE-K-K

#### 4.1 MONITORAMENTO

O monitoramento envolve a captura de propriedades do ambiente que são relevantes para as propriedades Self-X do sistema. São identificados dois tipos de monitoramento em sistemas autônomos [6]:

**Monitoramento passivo:** Realizado sem modificar o sistema monitorado. No Linux, por exemplo, comandos como `top` retornam informações sobre a utilização de CPU por cada processo, enquanto `vmstat` fornece estatísticas de memória e CPU. O diretório `/proc` é um sistema de arquivos virtual que contém informações de tempo de execução como memória

do sistema, informações de CPU, configuração de hardware, entre outros. Ferramentas similares existem para outros sistemas operacionais.

**Monitoramento ativo:** Envolve a instrumentação do software — a modificação ou adição de código à implementação da aplicação ou do sistema operacional para capturar chamadas de função ou de sistema. Ferramentas especializadas como o ProbeMeister são capazes de inserir probes diretamente no bytecode Java compilado, automatizando esse processo.

#### 4.2 PLANEJAMENTO E ADAPTAÇÃO

O Gerenciador Autônomo utiliza os dados coletados pelos sensores e o conhecimento interno do sistema para analisar o estado do elemento gerenciado, planejar as ações necessárias e executá-las por meio dos efetores [6]. Os objetivos do gerenciador são normalmente expressos por meio de políticas, que podem assumir três formas principais:

**Políticas ECA (*Event-Condition-Action*):** Definem ações diretas a serem tomadas quando um evento ocorre e uma condição é satisfeita. Por exemplo: “quando 95% dos servidores web apresentar tempo de resposta superior a 2 segundos e houver recursos disponíveis, aumentar o número de servidores ativos” [6]. Sua simplicidade é sua maior vantagem. Contudo, quando múltiplas políticas são especificadas, podem surgir conflitos entre regras que são difíceis de detectar antecipadamente e que só se manifestam em tempo de execução.

**Políticas de objetivo (*Goal Policies*):** Especificam critérios que caracterizam estados desejáveis do sistema, deixando ao próprio sistema a tarefa de determinar como atingi-los. São mais expressivas que as políticas ECA, mas exigem maior capacidade de raciocínio e planejamento por parte do gerenciador [6].

**Funções de utilidade (*Utility Functions*):** Definem quantitativamente o grau de desejabilidade de cada estado do sistema, atribuindo um valor numérico a cada combinação de parâmetros observáveis. Permitem ao gerenciador tomar decisões otimizadas mesmo quando o estado ideal não pode ser alcançado, identificando o estado menos indesejável entre as opções disponíveis [6].

Uma abordagem mais sofisticada de planejamento é o *modelo arquitetural dirigido*, no qual o gerenciador mantém um modelo da arquitetura do sistema gerenciado. Este modelo é atualizado continuamente pelos dados dos sensores e utilizado para planejar adaptações. A vantagem central dessa abordagem é a possibilidade de verificar, antes da execução, se a mudança planejada preserva a integridade do sistema [6].



### 4.3 BASE DE CONHECIMENTO

O componente de *Conhecimento* fundamenta todas as fases do laço MAPE-K. Ele pode ser tão simples quanto um conjunto de regras estáticas definidas por especialistas humanos, ou tão sofisticado quanto modelos preditivos construídos a partir de técnicas de aprendizado de máquina [1, 6]. Entre as abordagens mais relevantes, destacam-se:

**Aprendizado por reforço:** Aprende políticas de gerenciamento a partir da observação de ações em diferentes estados do sistema, avaliando as consequências de cada ação. Sua principal vantagem é não exigir um modelo explícito do sistema gerenciado, sendo adequado para ambientes altamente dinâmicos. A escalabilidade em espaços de estados grandes, no entanto, é um desafio reconhecido [6].

**Redes Bayesianas:** Utilizadas para a seleção probabilística de algoritmos e para a classificação sensível a custo de estados do sistema, auxiliando o gerenciador na tomada de decisões sob incerteza [6].

**Modelos arquiteturais:** Representações estruturais do sistema gerenciado, usadas para planejar adaptações e verificar a validade das mudanças antes de sua execução no sistema real, conforme descrito na seção anterior [1, 6].

A distinção entre o componente de planejamento e o de conhecimento é frequentemente tênue na prática: o conhecimento acumulado sobre o estado e o histórico do sistema gerenciado é o que torna possível ao gerenciador construir planos de adaptação confiáveis [6].

## 5 APLICAÇÕES

### 5.1 NASA *Autonomous NanoTechnology Swarm* (ANTS)

O projeto *Autonomous NanoTechnology Swarm* (ANTS) foi uma iniciativa da NASA desenvolvida no Centro de Voo Espacial Goddard, em colaboração com a Universidade de Ulster e a *Science Applications International Corporation* (SAIC). O projeto partia de uma premissa motivada tanto por limitações físicas quanto operacionais: nas missões de exploração do espaço profundo, o atraso de comunicação entre uma sonda e o controle em Terra pode chegar a dezenas de minutos em cada sentido, tornando inviável qualquer forma de supervisão humana em tempo real. As missões anteriores, como a DS1 (*Deep Space 1*) e a Mars Pathfinder, já haviam explorado graus limitados de autonomia embarcada; o ANTS representou um salto conceitual além desses precedentes, propondo um sistema inteiramente distribuído e sem controle centralizado em Terra [6, 7].

O conceito central do ANTS era uma frota de mil nanoespaçonaves da classe pico — cada uma com massa inferior a um quilograma — encarregadas de prospectar o cinturão de asteroides em busca de recursos minerais e compostos com relevância astrobiológica. Propulsionadas por velas solares, as espaçonaves seriam lançadas a partir de uma órbita no ponto de Lagrange L1 do sistema Terra-Lua. A frota organizava-se em três papéis funcionais complementares: *trabalhadores* (aproximadamente 80% da frota), responsáveis pela coleta de dados científicos com instrumentos especializados como magnetômetros e espectrômetros; *coordenadores*, encarregados de definir prioridades de missão e organizar o esforço coletivo; e *mensageiros*, responsáveis pela comunicação entre os diferentes subgrupos e com a estação terrestre [7]. O projeto previa dez tipos de especialistas distribuídos em dez subgrupos de aproximadamente cem espaçonaves cada.

Do ponto de vista da computação autônoma, o ANTS materializou todas as quatro propriedades Self-X descritas na Seção 3. A **auto-cura** era a propriedade mais crítica: dado que entre 60% e 70% das espaçonaves eram esperadas a ser perdidas ao entrar no cinturão de asteroides, as sobreviventes precisavam reorganizar-se autonomamente para redistribuir papéis e cobrir as lacunas deixadas pelas naves perdidas [6]. A **auto-configuração** manifestava-se na capacidade de cada espaçonave de determinar sua posição na frota, estabelecer rotas de comunicação com os vizinhos disponíveis e assumir um papel diferente quando necessário. A **auto-otimização** operava na alocação dinâmica de recursos da missão — decidindo quais asteroides priorizar com base nos dados já coletados pelos trabalhadores. Já a **auto-proteção** incluía mecanismos para evitar colisões entre naves do enxame e gerenciar autonomamente o consumo de energia das velas solares.

O modelo de controle do ANTS pode ser interpretado como uma

implementação distribuída do laço MAPE-K (Seção 4): cada espaçonave executava localmente seu próprio ciclo de monitoramento-análise-planejamento-execução, enquanto o comportamento coletivo emergente do enxame realizava um segundo nível de auto-gerenciamento de escopo global. Esse caráter hierárquico e emergente distinguia o ANTS dos projetos precursores, que dependiam de agentes individuais com comportamento predefinido. O desafio central residia em garantir que comportamentos corretos no nível local produzissem resultados coerentes e úteis no nível global — um problema que aproximava o projeto da pesquisa em sistemas multiagentes e inteligência de enxame [7].

## 5.2 IBM *Project eLiza*: SERVIDORES AUTO-GERENCIÁVEIS

A publicação do manifesto da IBM em outubro de 2001 levantou uma questão imediata: como traduzir os princípios de auto-gerenciamento em produtos comerciais concretos? A resposta foi o *Project eLiza*, iniciativa lançada como o maior foco individual de investimento na divisão de servidores da empresa à época [8]. O projeto estabelecia a linha *eServer* da IBM — que incluía as famílias zSeries (mainframes), pSeries, iSeries e xSeries — como plataforma de referência para a implementação progressiva das propriedades Self-X. A meta declarada era transformar a infraestrutura de TI corporativa em um sistema autônomo capaz de gerenciar a si próprio, reduzindo drasticamente a carga de administração humana em ambientes de alta complexidade [3, 8].

O *Project eLiza* introduziu capacidades autônomas concretas nos servidores zSeries, que já constituíam a espinha dorsal de grande parte das operações bancárias, de telecomunicações e governamentais globais. A **auto-configuração** foi implementada por meio de mecanismos que permitiam ao hardware e ao *middleware* reconfigurar subsistemas e recursos de forma autônoma tanto na inicialização quanto em tempo de execução, sem intervenção do operador [8]. A **auto-otimização** materializou-se no *Intelligent Resource Director* (IRD), componente do z/OS que monitorava continuamente a utilização de recursos e redistribuía cargas de trabalho entre partições lógicas (*LPARs*) para maximizar o *throughput* e o cumprimento dos acordos de nível de serviço [8].

Para a **auto-cura**, o projeto introduziu o *System Automation for OS/390*, uma camada de política baseada em regras capaz de detectar falhas em aplicações e recursos de sistema e acionar recuperações automáticas — iniciando, parando, monitorando e recuperando serviços z/OS sem intervenção humana [8]. Complementarmente, o *msys for Operations* automatizava tarefas operacionais rotineiras, reduzindo a dependência de procedimentos manuais suscetíveis a erro humano. A **auto-proteção** integrava mecanismos de detecção de intrusão e isolamento de falhas diretamente ao sistema operacional, garantindo a integridade de aplicações críticas diante de ameaças externas ou degradação interna de componentes.

O *Project eLiza* foi simultaneamente um produto comercial e uma prova de conceito científica. A IBM dedicou o volume 42 do *IBM Systems Journal*, publicado em 2003, integralmente ao tema da computação autônoma, reunindo artigos de pesquisadores da empresa sobre os fundamentos teóricos e as implementações práticas do projeto [8]. Ao demonstrar que as propriedades Self-X podiam ser incorporadas em sistemas de missão crítica em produção, o *Project eLiza* estabeleceu que a computação autônoma não era apenas uma visão acadêmica, mas uma abordagem de engenharia viável para infraestruturas reais — pavimentando o caminho para a gestão autônoma de larga escala que viria a caracterizar a era subsequente da computação em nuvem.

### 5.3 GERENCIAMENTO DE INFRAESTRUTURAS DE CLOUD COMPUTING

A ascensão da computação em nuvem consolidou o modelo de entrega de recursos como serviço (*Everything as a Service - XaaS*), exigindo infraestruturas capazes de suportar flutuações drásticas de demanda com latência mínima. Nesse contexto, a Computação Autônoma atua como o motor de orquestração essencial para manter a viabilidade econômica e operacional dos provedores de nuvem. O gerenciamento de um *Data Center* moderno envolve milhares de servidores físicos e milhões de máquinas virtuais (VMs) ou containers, onde decisões sobre alocação de recursos precisam ser tomadas em milissegundos [1].

A propriedade de auto-otimização é a mais visível nesta aplicação. Através do ciclo MAPE-K, gerentes autônomos monitoram métricas de desempenho como utilização de CPU, consumo de memória e tráfego de rede. Quando a carga de trabalho de uma aplicação atinge um limiar crítico, o sistema inicia um processo de *auto-scaling*, provisionando novos recursos sem intervenção humana para garantir o cumprimento dos Acordos de Nível de Serviço (SLAs). Inversamente, em períodos de baixa demanda, o sistema consolida as cargas de trabalho em um número menor de servidores físicos e desliga os equipamentos ociosos, resultando em uma economia significativa de energia e redução de custos operacionais [2].

Além da eficiência energética, a auto-cura é vital para a resiliência da nuvem. Em sistemas de larga escala, falhas de hardware são eventos estatisticamente certos. Gerentes autônomos detectam a degradação de componentes ou a falha total de um nó e, automaticamente, migram as VMs afetadas para servidores saudáveis, reconfigurando as tabelas de roteamento e balanceadores de carga de forma transparente para o usuário final [5]. Este nível de automação permite que a infraestrutura mantenha a alta disponibilidade mesmo sob condições de estresse ou falhas parciais.

#### 5.4 SISTEMAS PERVASIVOS E INTERNET DAS COISAS (IoT)

Se no modelo de nuvem a complexidade reside na densidade de recursos, nos sistemas pervasivos e na Internet das Coisas (IoT), o desafio reside na heterogeneidade e na volatilidade das conexões. Cidades inteligentes, redes elétricas inteligentes (*Smart Grids*) e ambientes hospitalares conectados dependem de uma miríade de dispositivos sensores e atuadores com capacidades computacionais limitadas e fontes de energia finitas [4].

A auto-configuração é o pilar fundamental para a escalabilidade da IoT. Em uma *Smart City*, novos sensores de monitoramento de tráfego ou qualidade do ar são adicionados à rede diariamente. Um sistema autônomo permite que esses dispositivos, ao serem ligados, descubram vizinhos, estabeleçam protocolos de comunicação seguros e registrem seus serviços no diretório central de forma plug-and-play, mimetizando a maneira como novas células se integram ao corpo humano [2]. Sem essa capacidade, o custo de instalação e configuração individual de milhões de dispositivos tornaria esses projetos financeiramente proibitivos.

Adicionalmente, a auto-proteção ganha uma nova dimensão em sistemas pervasivos. Devido à natureza distribuída e, muitas vezes, fisicamente exposta dos dispositivos IoT, eles são alvos frequentes de ataques de negação de serviço distribuído (DDoS) ou tentativas de infiltração. Gerentes autônomos de segurança utilizam modelos de inteligência artificial para identificar padrões de tráfego anômalos em tempo real. Ao detectar um comportamento suspeito em um grupo de sensores, o sistema pode isolar logicamente esses componentes do restante da rede (*quarentena*), impedindo a propagação de *malware* ou a exfiltração de dados sensíveis antes mesmo que os administradores humanos tomem conhecimento do incidente [1].

## 6 ESTADO DA ARTE: AVANÇOS, METODOLOGIAS E DESAFIOS

Desde a formulação do manifesto original da IBM [3], a Computação Autônoma (AC) transitou de uma visão teórica ambiciosa para um campo de pesquisa e desenvolvimento ativo, impulsionado pela necessidade de gerenciar ecossistemas de software cada vez mais complexos. O estado da arte atual não se concentra mais em provar a necessidade da autonomia, mas sim em refinar as metodologias subjacentes ao ciclo MAPE-K (*Monitor, Analyze, Plan, Execute over Knowledge*) e em resolver as limitações estruturais que impedem a adoção de sistemas totalmente independentes de intervenção humana.

### 6.1 PRINCIPAIS DESCOBERTAS E METODOLOGIAS

A principal mudança metodológica na última década foi a transição da *automação reativa* baseada em regras estáticas para a *autonomia preditiva* impulsionada por Inteligência Artificial (IA) e Aprendizado de Máquina (*Machine Learning* - ML). Historicamente, os gerentes autônomos dependiam de limiares pré-configurados (por exemplo, políticas do tipo *se-então*) para tomar decisões [5]. Embora eficazes em cenários previsíveis, essas abordagens falhavam em ambientes altamente dinâmicos e não-lineares.

Atualmente, a principal metodologia utilizada nas fases de Análise e Planejamento do ciclo MAPE-K envolve algoritmos de Aprendizado por Reforço (*Reinforcement Learning*) e redes neurais profundas. O sistema não recebe regras estáticas; em vez disso, o gerente autônomo aprende por tentativa e erro em ambientes simulados ou explora dados históricos para descobrir as melhores políticas de otimização de recursos [1].

Do ponto de vista aplicado, as principais descobertas consolidaram-se em dois grandes paradigmas da indústria:

- **AIOps (*Artificial Intelligence for IT Operations*):** Representa o estado da arte na auto-cura e auto-otimização de *data centers*. Ferramentas de AIOps ingerem terabytes de métricas e *logs* em tempo real, utilizando modelos preditivos para antecipar falhas de hardware ou picos de tráfego horas antes de ocorrerem, permitindo que o sistema atue preventivamente.
- **Orquestração Nativa da Nuvem e Malha de Serviços (*Service Mesh*):** A arquitetura de microsserviços adotou os princípios *Self-CH* de forma nativa. O uso integrado de orquestradores como Kubernetes com *mallas* de serviço permite a auto-configuração (descoberta automática de serviços) e auto-cura em nível de rede (redirecionamento de tráfego automático em caso de falha de um contêiner), operando na escala de

milissegundos.

## 6.2 LACUNAS E DESAFIOS EM ABERTO

Apesar dos avanços substanciais, a literatura aponta lacunas críticas que ainda separam o estado atual da visão de uma autonomia plena e universal [2, 4].

- 1 **Verificação Formal e Garantia de Estabilidade:** A introdução de algoritmos de ML no controle de infraestruturas críticas gerou o problema da "caixa-preta". Administradores hesitam em delegar o controle total ao sistema porque é difícil prever matematicamente como um modelo heurístico se comportará em situações anômalas. Atualmente, há uma lacuna significativa na aplicação de métodos formais que possam provar a corretude e a estabilidade das decisões autônomas. Pesquisas recentes buscam modelar o comportamento autônomo através de autômatos, cálculo de processos ou Redes de Petri para realizar a verificação formal das propriedades do sistema. O desafio é garantir que as ações de adaptação do gerente autônomo nunca levem o sistema a estados inseguros ou inconsistentes, um requisito mandatório para a adoção em sistemas de tempo real, como redes elétricas inteligentes ou veículos autônomos [1].
- 2 **Interoperabilidade e Sistemas Multi-agente:** A maioria das soluções atuais de computação autônoma atua em silos tecnológicos, desenvolvidos por um único provedor para um ambiente específico (como as bases de dados autônomas ou os balanceadores de carga proprietários). Quando ecossistemas heterogêneos precisam interagir, a falta de protocolos padronizados de comunicação para a troca de conhecimento (*Knowledge*) entre diferentes gerentes autônomos torna-se um gargalo [5]. A negociação e coordenação entre múltiplos elementos autônomos — para evitar que as otimizações locais de um gerente degradem o desempenho global do sistema — continua sendo um problema complexo de teoria dos jogos e sistemas multi-agente ainda não resolvido de forma definitiva.
- 3 **Tradução de Políticas de Alto Nível:** Outra lacuna prática reside na interface entre o administrador humano e o sistema autônomo. O objetivo da Computação Autônoma é permitir que humanos estabeleçam apenas as metas de negócios e SLAs. Contudo, a tradução semântica de um objetivo de alto nível (por exemplo, "maximizar o lucro minimizando a latência") para ações operacionais de baixo nível de configuração de rede e alocação de CPU continua exigindo, em grande medida, o conhecimento especializado do engenheiro [2].

Em suma, enquanto a capacidade de adaptação em tempo real atingiu

maturidade comercial, o estado da arte acadêmico concentra-se em resolver a crise de confiança. A fronteira da pesquisa reside em tornar o comportamento autônomo não apenas inteligente, mas formalmente verificável, interpretável e interoperável.



## 7 CONCLUSÃO

A análise do paradigma da Computação Autônoma revela que a indústria de Tecnologia da Informação atingiu um ponto de inflexão onde a escalabilidade dos sistemas não pode mais ser dissociada da automação de sua gestão. A evolução histórica, que partiu de sistemas isolados para ecossistemas hiper-conectados em nuvem e dispositivos de Internet das Coisas (IoT), evidenciou que a capacidade cognitiva humana é o principal limitador para a operação segura e eficiente das infraestruturas contemporâneas. A “crise da complexidade”, discutida desde o manifesto original da IBM [3], consolidou-se como o desafio central para engenheiros e pesquisadores da área.

Ao longo deste trabalho, observou-se que a implementação das propriedades *Self-CH*: auto-configuração, auto-cura, auto-otimização e auto-proteção, são fundamentadas na maturidade do ciclo de controle MAPE-K. Este modelo permite que o conhecimento do domínio seja codificado em gerentes autônomos capazes de agir proativamente frente a falhas ou flutuações de demanda. As aplicações discutidas, tanto no contexto de *Cloud Computing* quanto em sistemas pervasivos, demonstraram que a autonomia não é apenas uma questão de conveniência operacional, mas de viabilidade econômica. A capacidade de reduzir o custo total de propriedade (TCO) e mitigar o erro humano é o que permite a sustentabilidade de serviços globais que operam sob rigorosos Acordos de Nível de Serviço (SLAs) [4, 5].

Entretanto, o caminho para a plena autonomia ainda enfrenta barreiras significativas. A transição de sistemas que apenas auxiliam a decisão humana para sistemas que agem de forma totalmente independente exige avanços em termos de interoperabilidade e, crucialmente, de confiança. A estabilidade de algoritmos de controle em ambientes altamente dinâmicos e a garantia de que políticas de alto nível não resultem em estados sistêmicos indesejados permanecem como fronteiras de pesquisa ativa. Nesse sentido, a convergência da Computação Autônoma com técnicas de verificação formal e métodos rigorosos de modelagem apresenta-se como uma tendência promissora para assegurar que a auto-gestão seja tão previsível quanto eficiente [1, 2].

Em conclusão, a Computação Autônoma representa uma mudança fundamental na filosofia de design de software: passamos de sistemas que “fazem o que mandamos” para sistemas que “fazem o que queremos”. À medida que avançamos para uma era de trilhões de dispositivos interconectados, a inteligência intrínseca proposta por este paradigma deixará de ser um diferencial competitivo para se tornar um requisito mandatório. O sucesso dessa jornada garantirá que a tecnologia continue a evoluir como uma extensão invisível e resiliente da atividade humana, capaz de se regenerar, se proteger e se otimizar de forma transparente e segura.

## REFERÊNCIAS

- [1] Philippe Lalanda, Julie A. McCann, and Ada Diaconescu. *Autonomic Computing: Principles, Design and Implementation*. Undergraduate Topics in Computer Science. Springer, 2013. doi: 10.1007/978-1-4471-5007-7.
- [2] Jeffrey O. Kephart and David M. Chess. The vision of autonomic computing. *Computer*, 36(1):41–50, 2003.
- [3] IBM. Autonomic computing: Ibm’s perspective on the state of information technology. Technical report, IBM Corporation, Armonk, NY, October 2001. URL <http://www.ibm.com/research/autonomic>.
- [4] Roy Sterritt. Autonomic computing. *Innovations in Systems and Software Engineering*, 1(1):79–88, 2005.
- [5] Sand Corrêa and Renato Cerqueira. Computação autônoma: Conceitos, infra-estruturas e soluções em sistemas distribuídos. In *Sistemas Distribuídos: Redes de Computadores e Sistemas Distribuídos*, chapter 4. SBC, 2007.
- [6] Julie A. McCann Markus C. Huebscher. A survey of autonomic computing—degrees, models, and applications, 2008.
- [7] Walt Truszkowski, Mike Hinchey, James Rash, and Christopher Rouff. Nasa’s swarm missions: The challenge of building autonomous software. *IT Professional*, 6(5):47–52, 2004. NASA Technical Reports Server: 20040171635.
- [8] Alan G. Ganek and Thomas A. Corbi. The dawning of the autonomic computing era. *IBM Systems Journal*, 42(1):5–18, 2003. doi: 10.1147/sj.421.0005.